

# Công cụ mạnh của lời giải lặp: tối ưu hóa và ứng dụng thuật toán SPFA

Jiang Biye

Trường Trung học Kỹ niệm Tôn Trung Sơn, Quảng Đông

Luận văn Trại đông 2009

*Nguồn gốc: Công cụ mạnh của lời giải lặp: tối ưu hóa và ứng dụng thuật toán SPFA*

IOI China candidate team papers / 2009 / Jiang Biye

## Abstract

SPFA, tên đầy đủ là *Shortest Path Faster Algorithm*, là một cải tiến của thuật toán Bellman–Ford. Thuật toán dựa trên bất đẳng thức tam giác, và khi cài đặt thường dùng hàng đợi hoặc ngăn xếp để lặp liên tục cho tới khi thu được nghiệm tối ưu. Nó có ưu điểm là hiệu quả cao, cài đặt ngắn gọn và dễ mở rộng. Do bất đẳng thức tam giác có tính phổ quát, còn cách cài đặt lại gần với tìm kiếm, phạm vi dùng SPFA không chỉ giới hạn ở bài toán đường đi ngắn nhất trong lý thuyết đồ thị. Nó còn có thể phát huy tác dụng trong quy hoạch động và trong phương pháp lặp giải phương trình, xử lý một số bài toán không điển hình, đồng thời có thể được tối ưu theo từng bài cụ thể. Bài viết phân tích, thử nghiệm và thảo luận sâu các khía cạnh đó.

**Từ khóa.** Lặp; SPFA; tìm kiếm theo chiều sâu; tính tối ưu; quy hoạch động; chuyển trạng thái; phương trình.

## Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Giới thiệu thuật toán SPFA</b>                                 | <b>2</b>  |
| 1.1      | Cài đặt cơ bản của SPFA . . . . .                                 | 2         |
| 1.2      | Vận dụng linh hoạt: cài đặt SPFA bằng DFS và các tối ưu . . . . . | 4         |
| 1.2.1    | Nguyên lý cơ bản của SPFA dựa trên DFS . . . . .                  | 4         |
| 1.2.2    | Các tối ưu cho SPFA dựa trên DFS . . . . .                        | 4         |
| 1.3      | Thử nghiệm và so sánh hiệu quả thực tế . . . . .                  | 5         |
| 1.4      | Giới thiệu thuật toán Johnson . . . . .                           | 6         |
| <b>2</b> | <b>Tối ưu SPFA trong ứng dụng thực tế</b>                         | <b>7</b>  |
| 2.1      | Dùng SPFA để tìm nhanh chu trình âm hoặc dương . . . . .          | 7         |
| 2.2      | Chú ý cắt bỏ trạng thái vô dụng . . . . .                         | 9         |
| <b>3</b> | <b>Ứng dụng của thuật toán SPFA</b>                               | <b>10</b> |
| 3.1      | Hệ ràng buộc hiệu . . . . .                                       | 10        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.2      | Một lớp quy hoạch động có giai đoạn chuyển trạng thái không rõ . . . . . | 10        |
| 3.3      | Bàn về ứng dụng SPFA trong giải phương trình . . . . .                   | 11        |
| 3.4      | Một lớp quy hoạch động có chuyển trạng thái mang tính hậu hiệu . . . . . | 13        |
| <b>4</b> | <b>Phụ lục</b>                                                           | <b>16</b> |
| 4.1      | Nguyên văn ý chính các bài ví dụ . . . . .                               | 16        |
| 4.2      | Chương trình nguồn và dữ liệu thử trong gói gốc . . . . .                | 18        |
| <b>5</b> | <b>Tài liệu tham khảo</b>                                                | <b>18</b> |
| <b>6</b> | <b>Lời cảm ơn</b>                                                        | <b>18</b> |

# 1 Giới thiệu thuật toán SPFA

## 1.1 Cài đặt cơ bản của SPFA

Trước hết ta nhắc lại nguyên lý và điều kiện áp dụng của SPFA và Bellman–Ford. Tính chất quan trọng nhất là bất đẳng thức tam giác.

Giả sử  $G = (V, E)$  là một đồ thị có hướng có trọng số, hàm trọng số  $w : E \rightarrow \mathbb{R}$ ,  $s$  là đỉnh nguồn, và  $d(s, t)$  là độ dài đường đi ngắn nhất từ  $s$  tới  $t$ . Khi đó với mọi cạnh  $(u, v) \in E$ ,

$$d(s, v) \leq d(s, u) + w(u, v).$$

Nếu đặt  $d(s, s) = 0$ , bất đẳng thức này là điều kiện cần và đủ để  $d(s, t)$  là đường đi ngắn nhất từ  $s$  tới  $t$ .

Tính chất này rất dễ hiểu, và cũng dễ mở rộng. Giả sử  $G = (V, E)$  là một đồ thị có hướng có trọng số,  $d(u)$  là giá trị tối ưu của trạng thái hoặc đỉnh  $u$ , còn hàm trọng số  $w : E$  nhận giá trị trong một tập tự định nghĩa. Khi đó với mọi cạnh  $(u, v) \in E$ , ta có thể yêu cầu

$$d(u) + w(u, v) \text{ không tốt hơn } d(v).$$

Ở đây phép “+” có thể được mở rộng thành bất kỳ biểu thức tính toán nào. Hơn nữa, ta không nhất thiết phải giới hạn bất đẳng thức trong khuôn khổ “tính tối ưu”; tiêu chuẩn so sánh có thể do bài toán quyết định. Bất đẳng thức tam giác sau khi tổng quát hóa không còn bị ràng buộc vào đường đi ngắn nhất, mà có không gian áp dụng rộng hơn nhiều. Chỉ cần xây dựng được hàm trọng số giữa các trạng thái và tiêu chuẩn tốt/xấu phù hợp, trong phần lớn trường hợp ta có thể dùng SPFA để giải.

Khi đã có bất đẳng thức tam giác, thao tác lõi của SPFA, cũng là của Bellman–Ford, xuất hiện: **nới lỏng** (*relaxation*). Đặt  $f(u)$  là giá trị đường đi ngắn nhất hiện tại của đỉnh  $u$ , khởi tạo  $f(s) = 0$ ,  $f(u) = +\infty$  với  $u \neq s$ . Với mỗi cạnh chưa thỏa bất đẳng thức tam giác, ta gán lại giá trị cho đỉnh đích:

```
Relax(u, v):
    if f[v] > f[u] + w(u, v):
        f[v] = f[u] + w(u, v)
```

Bản chất của nới lỏng là liên tục tìm mâu thuẫn giữa trạng thái hiện tại và trạng thái đích, rồi điều chỉnh; khi không còn mâu thuẫn nào nữa, ta đạt tới trạng thái tối ưu.

Từ thao tác nới lỏng ta có vài hệ quả hữu ích.

**Tính chất cận trên và tính hội tụ.** Trong suốt thuật toán, với mọi  $u$  đều có  $f(u) \geq d(s, u)$ . Vì vậy một khi  $f(u)$  đã đạt tới  $d(s, u)$ , nó sẽ không đổi nữa. Điều này suy ra trực tiếp bằng quy nạp: nếu ban đầu  $f(u) \geq d(s, u)$ , sau khi nối lỏng cạnh  $(u, v)$  ta vẫn có

$$f(v) = f(u) + w(u, v) \geq d(s, u) + w(u, v) \geq d(s, v).$$

Khi không có chu trình âm, mỗi  $f(u)$  đơn điệu không tăng, do đó thuật toán nhất định dừng.

Nếu dùng cách tính Bellman–Ford:

```
for i = 1 to n - 1:
    for each edge (u, v) in E:
        Relax(u, v)
```

thì có thể tính đường đi ngắn nhất trong thời gian  $\mathcal{O}(NM)$ . Đường đi ngắn nhất có nhiều nhất  $N - 1$  cạnh; bằng quy nạp, sau vòng ngoài thứ  $i$  ta đã tính được đường đi ngắn nhất dùng  $i$  cạnh, nên sau vòng  $i + 1$  ta thu được đường đi dùng  $i + 1$  cạnh. Khi không có chu trình âm, số cạnh trên đường đi ngắn nhất không quá  $N - 1$ ; do đó nếu ở lần lặp thứ  $N$  vẫn còn cập nhật được thì chắc chắn tồn tại chu trình âm.

Trên thực tế, Bellman–Ford thường rất chậm vì có nhiều thao tác thừa. Xét đồ thị là một chuỗi

$$S \xrightarrow{-1} A_1 \xrightarrow{-1} \dots \xrightarrow{-1} A_n \xrightarrow{-1} T.$$

Nếu dùng trực tiếp Bellman–Ford, mỗi lần lặp thực chất chỉ cập nhật thêm một đỉnh, còn những vòng lặp khác đều vô hiệu. Vì thế chỉ những đỉnh đã được cập nhật ở lần lặp trước mới có thể hữu ích cho lần lặp hiện tại. SPFA tận dụng điều này bằng hàng đợi: chỉ lưu các trạng thái hiện còn hữu ích, mỗi lần một đỉnh được cập nhật thì đưa nó vào hàng đợi chờ mở rộng; các đỉnh chưa được cập nhật không được xét. Vì tại mọi thời điểm hàng đợi có nhiều nhất  $N$  phần tử, ta thường dùng hàng đợi vòng.

Program SPFA:

```
Init_graph()
Push_Queue(s)
while head != tail:
    x = Pop_Queue()
    for each edge (x, v) in E:
        if f[v] > f[x] + w(x, v):
            f[v] = f[x] + w(x, v)
            if v is not in queue:
                Push_Queue(v)
            if v has entered the queue N times:
                report a negative cycle
                halt
Output_data()
```

Trong trường hợp thông thường, SPFA rất hiệu quả. Có thể chứng minh kỳ vọng độ phức tạp là  $\mathcal{O}(KM)$  với  $K < 2$ , nhưng chứng minh này chưa đủ chặt và không phổ quát vì tồn tại dữ liệu chuyên đánh phá. Vì vậy bài viết không đi sâu chứng minh, mà dùng thử nghiệm thực tế ở phần sau để kiểm chứng hiệu quả.<sup>1</sup>

<sup>1</sup>Xem luận văn năm 2006 của Yu Yuanming, *Thuật toán đường đi ngắn nhất và ứng dụng*.

## 1.2 Vận dụng linh hoạt: cài đặt SPFA bằng DFS và các tối ưu

### 1.2.1 Nguyên lý cơ bản của SPFA dựa trên DFS

Trong thuật toán trên, ta dùng một hàng đợi vòng để lưu những đỉnh cần tiếp tục lặp, nên cách cài đặt giống tìm kiếm theo chiều rộng. Vậy có thể dùng công cụ còn lại của tìm kiếm, tức tìm kiếm theo chiều sâu, hay không?

Ở cách BFS, mỗi khi mở rộng được một đỉnh mới, ta luôn đưa nó vào cuối hàng đợi. Nhược điểm là tính liên tục của quá trình lặp bị ngắt. Nếu dùng tư tưởng DFS, ta có thể tiếp tục mở rộng ngay từ đỉnh mới đó. Do vậy thuật toán có thể được viết thành những lời gọi đệ quy bắt đầu từ các đỉnh vừa được cải thiện.

Việc phát hiện chu trình âm cũng đơn giản hơn. Nếu tồn tại chu trình âm  $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_k \rightarrow a_1$ , khi thuật toán DFS từ một đỉnh nào đó trên chu trình, cuối cùng nó có thể quay lại chính đỉnh đang nằm trên ngăn xếp đệ quy. Chỉ cần một mảng phụ ghi đỉnh hiện có trong ngăn xếp là có thể phát hiện chu trình âm kịp thời.

```
void SPFA(node):
    instack[node] = true
    for each edge (node, v) in E:
        if dis[node] + edge(node, v) < dis[v]:
            dis[v] = dis[node] + edge(node, v)
            if not instack[v]:
                SPFA(v)
            else:
                report a negative cycle
                halt
    instack[node] = false
```

Hai cách cài đặt có cùng nguyên lý, và đều tối ưu Bellman–Ford bằng một ý tưởng chung: chỉ giữ lại trạng thái hữu ích. Nhưng hiệu quả của DFS ra sao? Trong thử nghiệm ban đầu, tác giả nhận thấy khi quan hệ topo của dữ liệu mạnh, BFS có ưu thế lớn. DFS thường lặp mù quáng quá sâu, tốn rất nhiều thời gian, thậm chí vượt xa giới hạn.

### 1.2.2 Các tối ưu cho SPFA dựa trên DFS

Dù DFS ban đầu chưa cho kết quả tốt, ta không nên bỏ qua nó. So với BFS, cấu trúc linh hoạt của DFS đem lại không gian tối ưu rộng hơn.

Ý tưởng tự nhiên đầu tiên là thay đổi thứ tự tìm kiếm, tức sắp xếp các cạnh theo một quy tắc nào đó. Ở đây quy tắc hiển nhiên là sắp cạnh theo độ tốt/xấu của trọng số, nhờ đó có thể có nghiệm ban đầu khá tốt. Tối ưu này cho kết quả tốt trên vài bộ dữ liệu, nhưng nhiều bộ khác vẫn vượt thời gian.

Sau đó tác giả liên tưởng tới cách dùng luồng ban đầu tham lam trong bài toán luồng mạng. Chất lượng nghiệm hiện tại ảnh hưởng rất lớn tới những lần lặp về sau; sự kém hiệu quả của DFS chủ yếu đến từ việc dùng nghiệm kém để cập nhật một vùng trạng thái lớn. Do đó ta có thể đặt vài hạn chế trong lần DFS đầu để nhanh chóng thu được một nghiệm tương đối tốt, rồi mới chạy DFS đầy đủ lần hai. Sau nhiều thử nghiệm trên dữ liệu ngẫu nhiên, tác giả thu được một cách khởi tạo tham lam khá tốt:

```
Function SPFA_Init(node): boolean
    for each edge (node, v) in E:
```

```

        if dis[node] + edge(node, v) < dis[v]:
            dis[v] = dis[node] + edge(node, v)
            SPFA_Init(v)
        return true
    return false

```

Main:

```

    for each node in V:
        while SPFA_Init(node):
            pass
    for each node in V:
        SPFA(node)

```

Nghĩa là với mỗi đỉnh, chỉ cập nhật một cạnh rồi thoát. Trên đồ thị ngẫu nhiên, thuật toán này thường truyền nhanh giá trị hiện tại chỉ sau một lần cập nhật. Hiệu quả tăng đáng kể, nhưng vẫn còn kém BFS.

Tác giả tiếp tục so sánh BFS và DFS. Ưu thế chính của BFS là trước khi một đỉnh ra khỏi hàng đợi, nó có thể lại được cập nhật và nhận nghiệm tốt hơn cho lần lặp tiếp theo. DFS thì thường dùng một nghiệm thử cấp để đi xuống rất sâu; một nghiệm thử cấp có thể gây cập nhật cấp  $\mathcal{O}(N)$ . Từ đó có thể liên tưởng tới tìm kiếm sâu dần. Kết hợp với tư tưởng nghiệm ban đầu tham lam, ta giới hạn độ sâu đệ quy, rồi dần dần nới giới hạn.

Sau tối ưu này, trên dữ liệu dạng lưới vốn không thuận lợi cho BFS, thời gian DFS chỉ còn khoảng một phần ba BFS. Trên một số dữ liệu ngẫu nhiên khác, hiệu quả tốt xấu tùy trường hợp; nhìn chung vẫn hơi kém BFS nhưng khoảng cách đã nhỏ. Độ sâu giới hạn ảnh hưởng tới thời gian. Tác giả tổng kết hai cách đặt giới hạn tương đối hiệu quả:

1. Lần lượt đặt giới hạn độ sâu là  $1, 2, 4, 8, 16, 32, 64, \dots, 2^k, \dots, +\infty$ .
2. Lần lượt đặt giới hạn độ sâu là  $20, 30, 40, 50, 60, 80, 100, \dots, +\infty$ .

Do DFS hiệu quả trên đồ thị lưới, tác giả áp dụng nó cho bài *Skivall* trên SPOJ. Một lời giải khả thi của bài này là luồng chi phí trên đồ thị lưới. Sau khi dùng DFS, chương trình qua toàn bộ dữ liệu trong hơn 3 giây. Trên dữ liệu bài *Travel* của USACO tháng 1 năm 2009, DFS cũng nhanh hơn BFS rất nhiều, thậm chí tốt hơn Dijkstra, dù điều này phụ thuộc vào đặc tính dữ liệu.

Tới đây, việc tối ưu DFS tạm kết thúc. Có thể thấy những tối ưu khả thi cho DFS còn xa mới hết; nhiều tính chất chưa được khai thác. Hơn nữa, các tối ưu trên hầu như không làm mất tính ngắn gọn của SPFA, nên những hướng như thêm hàng đợi ưu tiên hoặc heap vẫn đáng nghiên cứu.

### 1.3 Thử nghiệm và so sánh hiệu quả thực tế

Phần này dùng dữ liệu thực tế để thử nghiệm SPFA.

**Môi trường thử nghiệm.** CPU Genuine Intel T2300 1.66 GHz, bộ nhớ 512 MB. Bộ chấm: Weidong Qingcheng Tester 1.0. Bài thử là đường đi ngắn nhất từ một nguồn tới toàn bộ đỉnh. Thời gian chạy gồm cả thời gian đọc dữ liệu; xuất đã được tối ưu nên có thể bỏ qua. Trong bảng gốc, giá trị in đậm màu đỏ là thuật toán nhanh nhất của bộ đó.

**Dữ liệu ngẫu nhiên.**

| Dữ liệu                           | BFS    | DFS thô   | DFS tham lam | DFS sâu hạn chế | Dijkstra | Đọc   |
|-----------------------------------|--------|-----------|--------------|-----------------|----------|-------|
| Âm, $N = 500, M = 100000$         | 0.13s  | 48.03s    | 0.13s        | 0.13s           | sai      | 0.13s |
| Âm, $N = 200000, M = 500000$      | 2.22s  | > 50s     | 3.14s        | 3.38s           | sai      | 1.45s |
| Âm, $N = 1000000, M = 2000000$    | 5.12s  | > 50s     | 5.61s        | 7.24s           | sai      | 3.06s |
| Dương, $N = 200000, M = 500000$   | 2.43s  | > 50s     | 3.48s        | 4.43s           | 1.92s    | 1.24s |
| Dương, $N = 1000000, M = 2000000$ | 5.39s  | > 50s     | 6.99s        | 8.86s           | 5.69s    | 2.55s |
| Tổng dữ liệu dương                | 7.82s  | > 100s    | 10.47s       | 13.29s          | 7.61s    | 3.79s |
| Tổng                              | 15.29s | > 248.03s | 19.35s       | 24.04s          | N/A      | 8.43s |

Từ bảng trên có thể rút ra vài kết luận. Thứ nhất, trên đồ thị ngẫu nhiên, BFS rất hiệu quả; với đồ thị thưa  $N = 10^6$ , nó còn nhanh hơn Dijkstra. Thứ hai, một hiện tượng lạ là dữ liệu trọng số dương của tác giả lại chậm hơn dữ liệu trọng số âm. Nguyên nhân là khi sinh dữ liệu âm, tác giả cố ý tăng trị tuyệt đối trọng số để tránh chu trình âm, khiến nhiều cạnh trở thành vô dụng và số cạnh âm thực tế ít đi. Điều này cho thấy hiệu quả thực tế của SPFA gắn chặt với dữ liệu. Thứ ba, DFS chưa tối ưu rất kém; nghiệm ban đầu tham lam tốt hơn giới hạn độ sâu trên dữ liệu ngẫu nhiên, dù vẫn hơi kém BFS.

### Dữ liệu đặc biệt: đồ thị tuyến tính và đồ thị lưới.

| Dữ liệu                           | BFS    | DFS thô | DFS tham lam | DFS sâu hạn chế | Dijkstra | Đọc   |
|-----------------------------------|--------|---------|--------------|-----------------|----------|-------|
| Tuyến tính dương, $N = 25000$     | 1.51s  | > 50s   | > 50s        | 1.70s           | 0.08s    | 0.06s |
| Tuyến tính dương, $N = 100000$    | 22.31s | > 50s   | > 50s        | > 50s           | 0.20s    | 0.17s |
| Lưới dương, $N = 700 \cdot 700$   | 9.06s  | > 50s   | > 50s        | 2.91s           | 1.67s    | 1.25s |
| Lưới dương, $N = 1000 \cdot 1000$ | 22.16s | > 50s   | > 50s        | 6.06s           | 3.70s    | 2.53s |
| Tổng tuyến tính                   | 23.82s | > 100s  | > 100s       | > 51.70s        | 0.28s    | 0.23s |
| Tổng lưới                         | 31.22s | > 100s  | > 100s       | 8.97s           | 5.37s    | 3.78s |

Với dữ liệu tuyến tính, cả hai cách cài đặt SPFA đều không tốt. Loại dữ liệu này xuất hiện trong bài PKU 3377, và bài gốc có thể giải trực tiếp bằng quy hoạch động. Với dữ liệu lưới, SPFA DFS có giới hạn độ sâu vượt xa BFS và đã khá gần Dijkstra.

**Tiểu kết.** Hiệu quả thời gian của SPFA dạng lập thay đổi rất mạnh. Trong thử nghiệm, không thể nói chung rằng đồ thị thưa thì nhanh, đồ thị dày thì chậm; đồ thị thưa vẫn có thể kém như dữ liệu tuyến tính, còn đồ thị dày chưa chắc chậm. Điều quyết định là hình dạng đồ thị và phân bố trọng số. Trên dữ liệu ngẫu nhiên, BFS thường tốt hơn; trên đồ thị lưới hoặc vài dữ liệu đặc biệt như *USACO Jan09 Gold Travel*, DFS lại có ưu thế. Cách chọn cụ thể phải dựa vào bài toán.

## 1.4 Giới thiệu thuật toán Johnson

Vì nhiều người chưa quen với Johnson, và bước tiền xử lý của Johnson cần SPFA, ta giới thiệu ngắn gọn ở đây. Johnson chủ yếu dùng để tính đường đi ngắn nhất mọi cặp trên đồ thị thưa. Ý tưởng chính là đổi trọng số để biến đồ thị có trọng số âm thành đồ thị có trọng số không âm, rồi chạy Dijkstra  $N$  lần.

Các bước đổi trọng số:

1. Thêm một đỉnh  $S$ , nối từ  $S$  tới mọi đỉnh bằng cạnh trọng số 0.
2. Dùng SPFA tính đường đi ngắn nhất  $h(i)$  từ  $S$  tới mỗi đỉnh  $i$ .
3. Với mỗi cạnh  $(u, v)$ , đặt  $w'(u, v) = w(u, v) + h(u) - h(v)$ .

Độ phức tạp xấu nhất của bước này là  $\mathcal{O}(NM)$ .

Với một đường đi bất kỳ từ  $s$  tới  $t$ :

$$s, x_1, x_2, \dots, x_k, t,$$

độ dài gốc là

$$f(s, t) = w(s, x_1) + w(x_1, x_2) + \dots + w(x_k, t).$$

Sau khi đổi trọng số, độ dài là

$$\begin{aligned} f'(s, t) &= h(s) + w(s, x_1) - h(x_1) + h(x_1) + w(x_1, x_2) - h(x_2) \\ &\quad + \cdots + h(x_k) + w(x_k, t) - h(t) \\ &= h(s) + f(s, t) - h(t). \end{aligned}$$

Vì  $h(s) - h(t)$  là hằng số đối với cặp  $(s, t)$ , việc đổi trọng số không làm thay đổi đường đi ngắn nhất. Mặt khác, do  $h(i)$  là đường đi ngắn nhất từ  $S$  tới  $i$ , theo bất đẳng thức tam giác ta có

$$h(u) + w(u, v) \geq h(v),$$

nên  $w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$ . Vì vậy đồ thị sau biến đổi có trọng số không âm và có thể giải thuận tiện bằng Dijkstra.

Nếu dùng heap Fibonacci, tổng độ phức tạp là

$$\mathcal{O}(NM + N^2 \log N + NM) = \mathcal{O}(NM + N^2 \log N).$$

Nếu dùng heap nhị phân, độ phức tạp là

$$\mathcal{O}(NM + N^2 \log N + NM \log N) = \mathcal{O}(N(N + M) \log N).$$

Điểm cốt lõi của Johnson là kỹ thuật đổi trọng số: biến trọng số thành không âm mà vẫn giữ đường đi ngắn nhất. Việc vận dụng linh hoạt bất đẳng thức tam giác đóng vai trò quan trọng, và các phần sau sẽ còn thấy nhiều ứng dụng như vậy.

## 2 Tối ưu SPFA trong ứng dụng thực tế

### 2.1 Dùng SPFA để tìm nhanh chu trình âm hoặc dương

Tìm chu trình âm là một bài toán quan trọng trong đồ thị. Vì chu trình âm và chu trình dương về bản chất giống nhau, phần này xét chu trình dương theo nhu cầu của ví dụ. Cách phổ biến là dùng Bellman–Ford hoặc SPFA.

Khi chu trình dương tồn tại, Bellman–Ford chắc chắn có độ phức tạp  $\mathcal{O}(NM)$ , không thể chấp nhận khi dữ liệu lớn. SPFA đã tối ưu hơn Bellman–Ford, nhưng nếu dùng nguyên cách cài đặt cơ bản, trong trường hợp cực đoan vẫn có thể suy biến thành  $\mathcal{O}(NM)$ . Vì thế nhiều người bi quan rằng không thể vượt qua  $\mathcal{O}(NM)$  khi tìm chu trình dương.

Thực ra nhận định này còn cách rất xa cận dưới tốt nhất  $\mathcal{O}(M)$  cho việc tìm một chu trình dương, cũng như độ phức tạp kỳ vọng  $\mathcal{O}(KM)$  của SPFA trung bình. Điều đó gợi ý ta có thể tận dụng tính chất liên quan để tối ưu.

#### Ví dụ 1: WordRings

Ta có  $n$  xâu,  $n \leq 100000$ , mỗi xâu gồm các chữ cái thường từ a tới z. Nếu hai ký tự cuối của xâu  $A$  khớp với hai ký tự đầu của xâu  $B$ , ta nói  $A$  có thể nối với  $B$ ; chú ý chiều ngược lại chưa chắc đúng. Cần chọn một số xâu cho chúng nối đầu đuôi thành một vòng xâu, một xâu tự nối đầu đuôi cũng được, sao cho độ dài trung bình là lớn nhất. Ví dụ ababc, bckjaca, caahoynaab có thể nối thành vòng theo thứ tự đó, tổng độ dài  $5 + 7 + 10 = 22$ , dùng 3 xâu nên trung bình khoảng 7.33.

Bài toán kết hợp hai mô hình quen thuộc: nối xâu và tối ưu giá trị trung bình. Với nối xâu, ta xem mỗi từ là một cạnh, còn cặp hai chữ cái đầu/cuối là đỉnh. Chẳng hạn ababc là cạnh từ đỉnh ab tới đỉnh bc

có độ dài 5. Nhờ vậy quy mô giảm còn nhiều nhất  $26^2$  đỉnh. Bài toán trở thành tìm một chu trình có trung bình trọng số cạnh lớn nhất.

Với chu trình gồm các cạnh  $E_1, E_2, \dots, E_k$ ,

$$Average = \frac{E_1 + E_2 + \dots + E_k}{K}.$$

Do đó

$$(E_1 - Average) + (E_2 - Average) + \dots + (E_k - Average) = 0.$$

Ta nhị phân đáp án  $Ans$ , đổi trọng số mỗi cạnh thành  $E_i - Ans$ , rồi kiểm tra có tồn tại chu trình có tổng dương hay không. Nếu dùng cách thường, độ phức tạp là  $\mathcal{O}(NM \log Ans)$  với  $N \leq 26^2$ ,  $M \leq 100000$ , cả lý thuyết lẫn thực tế đều khó chấp nhận. Vì vậy phải tối ưu việc tìm chu trình dương.

**Các tối ưu.** Trước hết, khi dùng SPFA BFS để tìm chu trình dương, khác với tìm đường đi ngắn nhất, ta có thể khởi tạo mọi  $dis[node]$  bằng 0, rồi đưa tất cả đỉnh vào hàng đợi để lập. Điều kiện cần và đủ để phát hiện chu trình dương là có đỉnh vào hàng đợi quá  $N$  lần. Việc khởi tạo 0 không ảnh hưởng tính đúng đắn, và giảm mạnh số lần mở rộng. Bellman–Ford không chỉ chậm mà đôi khi còn tràn số: sau  $N$  vòng lặp, một đỉnh có thể bị cập nhật cấp  $NM$ , dẫn tới  $N \cdot M \cdot MaxEdge$  vượt giới hạn số nguyên.

1. Nếu giá trị hiện tại của một đỉnh lớn hơn  $n \cdot MaxEdge$ , chắc chắn tồn tại chu trình dương.
2. Đồ thị có hướng, chu trình dương chỉ có thể nằm trong một thành phần liên thông mạnh. Có thể tìm các thành phần liên thông mạnh và xóa các cạnh giữa các thành phần. Tối ưu này có tác dụng trên dữ liệu ngẫu nhiên, nhưng làm tăng độ phức tạp cài đặt và dễ bị dữ liệu toàn đồ thị liên thông mạnh đánh bại, nên không được khuyến nghị.
3. Nếu tổng số lần phần tử vào hàng đợi vượt quá  $T(M + N)$ , kết luận có chu trình dương.  $T$  có thể chọn theo bài, thường lấy  $T = 2$ . Tối ưu này hy sinh tính đúng đắn tuyệt đối, nhưng kinh nghiệm thực tế ủng hộ mạnh mẽ: khi không có chu trình dương, SPFA thường có kỳ vọng  $\mathcal{O}(KM)$ ; khi hiệu quả trở nên rất thấp, khả năng cao là vì có chu trình dương.

Những tối ưu trên đều xoay quanh BFS. Nếu dùng DFS ở phần 1 thì sao? Kết quả thử nghiệm khiến tác giả bất ngờ: trên PKU Online Judge, chương trình DFS chỉ dùng 219 ms để qua toàn bộ dữ liệu, nhanh hơn cả chương trình BFS có giới hạn số phần tử vào hàng đợi. Nguyên nhân là DFS luôn làm nhiều đỉnh được cập nhật rất nhanh; một khi DFS đi vào một đỉnh trên chu trình dương, nó rất có thể đi theo chu trình và quay lại điểm xuất phát. Khởi tạo 0 cũng giúp DFS tránh nhiều nhánh thừa. Trong mức độ lớn, thời gian của DFS gần  $\mathcal{O}(M)$ .

| Dữ liệu                                | BFS thô | BFS giới hạn $T = 2$ | DFS   | Độc   |
|----------------------------------------|---------|----------------------|-------|-------|
| Ngẫu nhiên, $N = 10000, M = 300000$    | > 50s   | 0.19s                | 0.16s | 0.16s |
| Ngẫu nhiên, $N = 200000, M = 500000$   | > 50s   | 4.34s                | 1.61s | 1.47s |
| Ngẫu nhiên, $N = 1000000, M = 2000000$ | > 50s   | 4.85s                | 3.79s | 3.00s |
| Một chu trình 200000 đỉnh              | > 50s   | 3.73s                | 1.55s | 1.41s |
| Tổng                                   | > 200s  | 13.11s               | 7.11s | 6.04s |

DFS qua tốt nhiều loại dữ liệu và chạy rất nhanh. BFS có giới hạn tuy đã hy sinh tính đúng đắn nhưng vẫn chậm hơn DFS, và gặp rủi ro lớn trước dữ liệu mạnh như dữ liệu tuyến tính.

Trong thử nghiệm, DFS cũng tốt cả khi có và không có chu trình dương. Điều này có thể làm người đọc băn khoăn: nếu không tìm thấy chu trình dương thì SPFA kết thúc, tức đã tính đường đi dài nhất, vậy tại sao DFS tìm đường đi ngắn nhất ở phần trước lại không hiệu quả như thế? Khác biệt là khi tìm chu trình, ta khởi tạo tất cả giá trị bằng 0, giảm rất nhiều tính toán vô dụng.

Tính đúng đắn của khởi tạo 0 có thể chứng minh như sau. Giả sử ban đầu tồn tại một đỉnh  $s$  từ đó ta có thể tìm chu trình dương, nghĩa là đi một vòng trên chu trình bắt đầu từ  $s$  thì mỗi lần qua một đỉnh  $x$ , giá trị  $dis[x]$  đều lớn hơn 0. Xét một đỉnh  $x$  trên chu trình, có tiền nhiệm  $y$ . Khi tìm chu trình, đáng lẽ  $dis[y] + w(y, x) > dis[x]$  và ta tiếp tục từ  $x$ . Nếu trước đó  $dis[x]$  đã được gán lại thành  $dis'[x] \geq dis[y] + w(y, x)$ , thì khi tới  $x$  từ  $y$  có vẻ bị chặn; nhưng chính lần gán lại đó đã cho phép ta bắt đầu tiếp từ  $x$  rồi, không cần đi qua  $y$ . Lập luận tương tự, luôn tìm được một điểm xuất phát dẫn tới chu trình dương. Giả thiết ban đầu cũng hiển nhiên: nếu không, ta có thể chia chu trình dương thành vài đoạn có tổng trọng số không dương, mâu thuẫn với tổng dương.

## 2.2 Chú ý cắt bỏ trạng thái vô dụng

Ở trên ta đã thấy hiệu quả của SPFA phụ thuộc nhiều vào chất lượng giá trị hiện tại. Thuật toán thường lặp nhiều trên các nghiệm thứ cấp, lãng phí thời gian. Ngoài những tối ưu đã nêu, trong các bài không phải đường đi ngắn nhất đơn giản mà có thêm điều kiện, ta thường có thể phân tích bài cụ thể để cắt bỏ trạng thái vô dụng.

### Ví dụ 2: đường đi hai tiêu chí

Mạng đường ngày nay ngày càng dày và phí đi lại ngày càng nhiều, nên chọn đường tốt nhất là một vấn đề thực tế. Đường giữa các thành phố là hai chiều; mỗi đường có thời gian đi và phí phải trả. Một tuyến đường tốt hơn tuyến khác nếu nó mất ít thời gian hơn và không đắt hơn, hoặc rẻ hơn và không chậm hơn. Một tuyến được gọi là tối thiểu nếu không có tuyến nào tốt hơn nó. Cho mạng giao thông, thành phố đầu và cuối, hãy tính số cặp phí-thời gian tối thiểu. Ràng buộc:  $N \leq 100$ ,  $M \leq 300$ , phí và thời gian trên mỗi cạnh không quá 100.

Có hai hướng đã được thảo luận nhiều: xây đồ thị phân tầng rồi giải đường đi ngắn nhất, hoặc dùng trực tiếp trạng thái hai chiều và SPFA. Ở đây xét hướng thứ hai. Vì có hai trọng số, tại mỗi đỉnh ta thêm một chiều trạng thái. Đặt  $F[i, j]$  là thời gian nhỏ nhất để tới đỉnh  $i$  với tổng phí đã dùng là  $j$ . Khi đó

$$F[i, j] = \text{Min}\{F[k, j - \text{Cost}(k, i)] + \text{Time}(k, i) \mid (k, i) \in E\}.$$

Ta có thể dùng SPFA trực tiếp, kỳ vọng thời gian  $\mathcal{O}(KN^2V)$ , với  $K$  là hằng số lặp. Số trạng thái có thể đạt cấp  $100^3$ ; dù chạy thực tế khá nhanh, vẫn nên tối ưu.

Xét một trạng thái mới  $F[i, j]$ . Nếu đã có trạng thái  $F[i, k]$  với  $k < j$  và  $F[i, k] < F[i, j]$ , thì  $F[i, j]$  chắc chắn không thể nằm trên biên tối thiểu và không cần đưa vào hàng đợi. Vì vậy, với mỗi trạng thái mới  $F[i, j]$ , ta truy vấn giá trị nhỏ nhất của  $F[i, 0..j]$ , gọi là  $\text{Timemin}$ ; chỉ khi  $\text{Timemin} > F[i, j]$  mới cập nhật. Có thể duy trì mỗi  $F[i]$  bằng cây Fenwick để truy vấn nhanh. Kỳ vọng lý thuyết thành  $\mathcal{O}(KN^2V \log N)$ , nhưng hiệu quả thực tế rất cao.

| Thuật toán                   | Tổng thời gian trên toàn bộ dữ liệu |
|------------------------------|-------------------------------------|
| SPFA chưa tối ưu             | 2.09s                               |
| SPFA tối ưu                  | 0.24s                               |
| Chương trình chuẩn dùng heap | 2.60s                               |

Tối ưu này rất đơn giản, tính chất cũng hiển nhiên, nhưng do quá trình lặp của SPFA có phần mù quáng, thực tế thường xuất hiện nhiều trạng thái vô dụng như vậy. Nếu chúng tích lũy nhiều, hiệu quả bị ảnh hưởng nghiêm trọng. Vì thế, phân tích bài cụ thể để loại bỏ trạng thái vô dụng kịp thời là rất quan trọng.

### 3 Ứng dụng của thuật toán SPFA

#### 3.1 Hệ ràng buộc hiệu

Ràng buộc hiệu là một lớp bài toán kinh điển dùng SPFA. Dạng chung là gán giá trị cho một số trạng thái sao cho các giá trị đó thỏa những quan hệ bất đẳng thức giữa trạng thái. Nội dung cụ thể đã được nhắc trong luận văn đội tuyển năm 2006 của Feng Wei, nên bài viết chỉ nêu một ví dụ.

##### Ví dụ 3: Segments

Trên mặt phẳng có  $N$  đoạn thẳng. Cần chọn một số đường thẳng vuông góc với trục  $X$  sao cho mỗi đoạn thẳng bị cắt ít nhất một lần và nhiều nhất  $R$  lần, không tính trường hợp cắt đúng đầu mút; yêu cầu tối thiểu hóa  $R$ .

Với bài tối thiểu hóa giá trị lớn nhất, ta thường nhị phân đáp án rồi biến thành bài kiểm tra. Sau khi nhị phân  $R$ , rồi rạc hóa các hoành độ. Vì cắt ở đầu mút không hợp lệ, các vị trí chọn đường thẳng có  $L \leq 2N$  lựa chọn, và mỗi đoạn thẳng tương ứng với một khoảng liên tiếp  $[a, b]$ . Nếu tại mỗi vị trí dùng 0/1 để biểu diễn có đặt đường thẳng hay không, ta cần kiểm tra có phương án sao cho số lượng 1 trong mỗi khoảng lớn hơn 0 và không vượt  $R$ . Trạng thái 0/1 trực tiếp khó viết bất đẳng thức, nên đặt  $S[i]$  là số lượng 1 trong các vị trí  $1..i$ . Khi đó:

$$S[0] = 0, \quad S[i-1] \leq S[i] \quad (1 \leq i \leq L),$$

và với mỗi đoạn  $[a, b]$ ,

$$1 \leq S[b] - S[a-1] \leq R.$$

Ta đã xây dựng được các quan hệ bất đẳng thức giữa các  $S[i]$ ; chỉ cần kiểm tra có chu trình âm hay không.

Ràng buộc hiệu là một mở rộng dựa trên bất đẳng thức tam giác. Mấu chốt là xây dựng trạng thái phù hợp theo đề bài, thiết lập bất đẳng thức giữa trạng thái, và chứng minh nghiệm thỏa bất đẳng thức tương ứng một-một với nghiệm của bài gốc. Sau đó có thể dùng SPFA. Ưu điểm là quan hệ bất đẳng thức rõ ràng, dễ hiểu; nhược điểm là phạm vi áp dụng hẹp.

#### 3.2 Một lớp quy hoạch động có giai đoạn chuyển trạng thái không rõ

Trong một số bài quy hoạch động, ta gặp tình huống trạng thái  $A$  có thể cập nhật từ trạng thái  $B$ , đồng thời  $B$  cũng có thể cập nhật từ  $A$ . Khi đó giữa các trạng thái xuất hiện chu trình, không thể tổ chức một quan hệ topo để chia giai đoạn.

Trong trường hợp thông thường, chu trình chỉ tồn tại trong phương trình chuyển trạng thái, còn trong nghiệm tối ưu sẽ không có chu trình. Điều này giống bài đường đi ngắn nhất: đồ thị có chu trình, nhưng các đường đi ngắn nhất tạo thành một cây đường đi. Những bài như vậy đôi khi có thể giải bằng phương pháp gán nhãn kiểu Dijkstra, hoặc tận dụng tính chất đặc biệt để dựng thứ tự topo. Nhưng Dijkstra không heap thì chậm, dùng heap thì cài đặt phức tạp; hơn nữa DP thường nằm trên đồ thị ẩn nên tổ chức trạng thái cho Dijkstra không thuận tiện. Thứ tự topo hiệu quả cũng có thể rất ẩn hoặc không tồn tại.

Đây là lúc SPFA có đất dụng võ. Cách cài đặt linh hoạt của nó tương thích dễ dàng với nhiều phương trình DP, và trong DP hiếm khi có dữ liệu chuyên đánh phá SPFA.

##### Ví dụ 4: Kế hoạch tham quan

Một học sinh tới Winter Camp 2008 ở Thiệu Hưng và muốn thăm mọi điểm tham quan. Ban tổ chức chia bản đồ thành lưới  $N$  hàng,  $M$  cột. Mỗi ô có nhiều nhất một điểm tham quan; ô không có điểm tham quan

là đường. Để bảo đảm an toàn, ban tổ chức bố trí tình nguyện viên ở một số ô đường; ở điểm tham quan thì thuê hướng dẫn viên. Cần chọn các ô sao cho giữa hai điểm tham quan bất kỳ có một đường đi mà mọi ô trên đường đều có tình nguyện viên hoặc là điểm tham quan, đồng thời tổng số tình nguyện viên là nhỏ nhất. Các ràng buộc đều không quá 10:  $N, M$  và số điểm tham quan.

Bài này có thể giải trực tiếp bằng quy hoạch động nén trạng thái dựa trên tính liên thông, nhưng cài đặt phức tạp. Phân tích thêm, ta thấy có nhiều nhất 10 điểm tham quan, và mỗi phần của một nghiệm là một cây. Cây biểu diễn nghiệm chỉ liên quan tới vị trí và tình trạng liên thông. Đặt  $f[i, j, k]$  là giá trị tối ưu của cây có gốc tại ô  $(i, j)$ , nối được tập điểm tham quan  $K$  (một mask 10 bit).

Khởi tạo: đánh số các điểm tham quan; nếu điểm thứ  $T$  nằm tại  $(i, j)$ , thì  $f[i, j, 2^{T-1}] = 0$ .

Có hai loại chuyển. Loại thứ nhất là hợp nhất hai nghiệm kề nhau:

$$f[i, j, t_1 \text{ or } t_2] \leftarrow \min(f[i, j, t_1 \text{ or } t_2], f[i, j, t_1] + f[i', j', t_2]),$$

trong đó  $(i, j)$  kề  $(i', j')$ . Loại thứ hai là mở rộng nghiệm hiện tại qua một ô đường:

$$f[i, j, t] \leftarrow \min(f[i, j, t], f[i', j', t] + \text{VolunteerNumber}[i, j]).$$

Cần chú ý phép hợp nhất có thể tạo hai ô trùng nhau, về mặt hình thức là không hợp lệ; nhưng luôn tồn tại một nghiệm không trùng có giá trị tương tự, nên trạng thái đó không thể trở thành nghiệm tối ưu. Thuật toán vẫn xét đủ mọi trường hợp tối ưu.

Vấn đề cài đặt nằm ở loại chuyển thứ hai. Nếu  $(i, j)$  kề  $(i', j')$ , ta có thể chuyển từ  $F[i, j, t]$  sang  $F[i', j', t]$ , và cũng có thể chuyển ngược lại; không có thứ tự rõ ràng. Có thể dùng gán nhãn kiểu Dijkstra, mỗi lần lấy trạng thái hiện tốt nhất để mở rộng, nhưng hơi rườm rà và không quá hiệu quả.

Thực ra bài này thỏa bất đẳng thức tam giác và tính chất cận trên đã nói: với mọi ô  $(i, j)$ ,

$$f[i, j, t] \leq f[i', j', t] + \text{VolunteerNumber}[i, j].$$

Giá trị khởi tạo cũng không nhỏ hơn nghiệm tối ưu. Có thể xem trực tiếp bài toán là đường đi ngắn nhất đa nguồn đa đích. Trước hết chia giai đoạn theo mask cây  $t$ , rồi với mỗi  $t$ , chạy một lần SPFA trên tất cả  $F[i, j, t]$ : liên tục tìm đỉnh có thể cập nhật, lấy nó làm điểm xuất phát mới và cập nhật các đỉnh kề. Khi kết thúc, toàn bộ trạng thái của mask đó đã tối ưu. Kỳ vọng thời gian là  $\mathcal{O}(KNM \cdot 3^T)$ , trong đó  $K$  là hằng số nhỏ,  $T$  là số điểm tham quan.

Đây là một ứng dụng rất thường gặp của SPFA trong bài toán tối ưu. Chỉ cần có phương trình chuyển đúng, giá trị cuối cùng của trạng thái thỏa bất đẳng thức tam giác, và bảo đảm nghiệm cuối không có chu trình, thì dù quan hệ chuyển có chu trình, SPFA vẫn giải được. Trong bài tối ưu thông thường, điều kiện không có chu trình thường được chứng minh bằng tính đơn điệu.

### 3.3 Bàn về ứng dụng SPFA trong giải phương trình

Giải phương trình, đặc biệt hệ phương trình tuyến tính, là một lớp bài thường gặp trong tin học. Mô hình của chúng đa dạng và áp dụng rộng; nhiều bài cuối cùng có thể quy về giải phương trình. Với hệ tuyến tính, cách giải phổ biến là khử Gauss, nhưng trong thực tế có thể phát sinh nghiệm không hợp lệ hoặc lỗi chính xác khi giải phương trình nguyên. Dưới đây ta thảo luận cách dùng SPFA trong một số hoàn cảnh. Kết quả không hoàn hảo: SPFA không thể thay thế hoàn toàn khử Gauss, và độ chính xác cần được trả bằng thời gian. Mục đích là đưa ra một hướng nghĩ khác.

#### Ví dụ 5: Mario

Mario ở trong bản đồ  $N \times M$ ,  $N, M \leq 15$ . Bản đồ gồm đất trống, tường, quái vật, ô tiền vàng (chứa 0 tới 9 đồng) và ống dịch chuyển. Tường không đi qua được; gặp quái vật thì mất 1 mạng; gặp ô tiền vàng thì

lấy được tiền nhưng tiền vẫn còn, lần sau tới vẫn lấy tiếp; đi vào cửa ống thì lập tức dịch tới đầu ra. Ban đầu Mario có 3 mạng. Mỗi bước Mario đi ngẫu nhiên sang một trong bốn hướng hợp lệ (không ra ngoài bản đồ và không vào tường). Hồi kỳ vọng số tiền lấy được trước khi trò chơi kết thúc, tức mạng bằng 0 hoặc vĩnh viễn không thể lấy thêm tiền; nếu kỳ vọng vô hạn thì báo vô hạn.

Với bài kỳ vọng, thường phải lập phương trình. Đặt  $f[x, y, life]$  là kỳ vọng số tiền nhận được khi Mario có  $life$  mạng và đang ở ô  $(x, y)$ . Phương trình là

$$f[x, y, life] = \sum \frac{f[x', y', life']}{Walknumber},$$

trong đó  $(x', y', life')$  là các trạng thái có thể đạt tới sau một bước;  $Walknumber$  là số trạng thái kế tiếp, thường là 4 nhưng giảm khi có vật cản. Cần xử lý điều kiện biên: khi Mario không thể tiếp tục lấy tiền, kỳ vọng là 0. Ngoài ra còn phải thảo luận các trường hợp vô nghiệm hoặc nghiệm không hợp lệ do phương trình và bài gốc không tương ứng hoàn toàn, nhưng đó không phải trọng tâm ở đây.

Xét một góc khác. Bài này có đặc điểm là không có hai phương trình gốc có cùng ẩn ở vế trái, nếu không xét việc chuyển vế. Nghĩa là không xuất hiện đồng thời

$$X = Y + Z, \quad X = U + W.$$

Điều này bảo đảm ta có thể gán lại các ẩn liên tục mà không xung đột. Hơn nữa, vì đây là bài kỳ vọng, các hệ số ở vế phải đều nhỏ hơn 1, nên giá trị lặp không phát tán quá mạnh.

Xét ví dụ nhỏ: Mario đang ở điểm  $x$ , chỉ có hai đường. Một đường khiến mất 1 mạng và trò chơi kết thúc; đường còn lại đi tới ngõ cụt  $y$  nhưng nhận được một đồng tiền. Ở  $y$  chỉ có thể quay lại. Ta có hai phương trình:

$$Gold(x) = \frac{0 + Gold(y) + 1}{2}, \quad Gold(y) = Gold(x).$$

Nghiệm là  $Gold(x) = Gold(y) = 1$ . Các phương pháp kiểu DP thông thường không giải trực tiếp được vì các trạng thái phụ thuộc vòng quanh nhau; nghiệm cuối không có cấu trúc topo hình cây như đường đi ngắn nhất.

Vì không thể lấy nghiệm đúng trong một lần, ta thử dùng SPFA để lặp tới nghiệm đúng. Khởi tạo  $Gold(x) = Gold(y) = 0$ . Lúc này  $(Gold(y) + 1 + 0)/2 = 0.5 \neq Gold(x)$ , nên gán lại  $Gold(x) = 0.5$ , đồng thời  $Gold(y) = 0.5$ . Lần hai được 0.75, lần ba 0.875, lần bốn 0.9375, v.v. Sau  $k$  lần lặp,

$$Gold(x) = Gold(y) = 1 - 2^{-k}.$$

Khoảng 30 lần lặp đã đạt độ chính xác cấp  $10^{-9}$ , và hội tụ đúng về nghiệm.

Từ ví dụ này, ta có cách giải tổng quát: khởi tạo mọi giá trị bằng 0 để bảo đảm không lớn hơn nghiệm cuối; đặt hằng số chính xác  $\varepsilon = 10^{-8}$  tùy yêu cầu bài; sau đó kiểm tra từng phương trình, thay giá trị lặp hiện tại vào vế phải. Nếu độ chênh giữa hai vế lớn hơn  $\varepsilon$  thì cập nhật, lặp tới khi mọi phương trình đều thỏa.

Cách này còn đúng khi số ẩn và số phương trình tăng không? Theo nguyên lý SPFA ở phần 1.1, tính chất quan trọng là mọi hệ số đều dương, nên trong quá trình lặp, giá trị hiện tại của mỗi ẩn đơn điệu không giảm. Mỗi giá trị luôn tiến tới một cận trên xác định mà không vượt qua cận đó; khi khoảng cách tới cận trên đủ nhỏ thì thuật toán dừng. Kết quả cuối thỏa hệ phương trình trong phạm vi sai số, nên thuật toán khả thi. Nếu kỳ vọng vô hạn, tức không tồn tại cận trên, thuật toán sẽ không dừng; ta cần phát hiện nó tương tự phát hiện chu trình âm.

Một câu hỏi tự nhiên là: bước tiến của dãy xấp xỉ càng lúc càng nhỏ, liệu quá trình lặp có bị đình trệ vì bước quá nhỏ hay không? Về mặt dãy số và giới hạn, các bước tiến tất nhiên giảm dần do hội tụ. Tuy nhiên trong tính toán thực, sai số số thực và số vòng lặp hữu hạn có thể gây vấn đề chính xác, như phân tích dưới đây.

**Cài đặt và hiệu quả.** Cài đặt hơi khác SPFA trước đó, vì khi gán giá trị cho một điểm (ẩn), nhiều điểm khác cùng tham gia. Lỗi vẫn là chỉ lưu trạng thái đang hoạt động.

Program Mario:

```
for each node x:
    Now[x] = 0
    Push(x)
while head != tail:
    x = Pop()
    for each edge (x, u) in E:
        if abs(Get(u) - Now[u]) > Eps:
            Now[u] = Get(u)
            Push(u)
```

Trong đó  $Get(u)$  tính giá trị về phải của phương trình ứng với  $u$ . Ta nổi cạnh từ  $x$  tới  $u$  khi  $x$  xuất hiện ở về phải phương trình của  $u$ . Khi cài đặt có thể xây hai đồ thị, một thuận và một nghịch.

Để nhận biết kỳ vọng vô hạn, có thể giới hạn tổng số lần vào hàng đợi như phần trên. Chú ý không dùng DFS, vì bản thân bài này là liên tục cập nhật theo chu trình; việc quay lại chính mình từ một điểm là bình thường. Một ưu điểm là SPFA không cần tiền xử lý đặc biệt cho nhiều trường hợp, vì nghiệm tìm được tự nhiên là hợp lệ, đơn giản hơn khử Gauss.

Kết quả thực tế về cơ bản đúng như dự đoán: phần lớn dữ liệu có thể đạt độ chính xác cao trong giới hạn thời gian. Nhưng với một số dữ liệu đặc biệt, nhất là có điểm dịch chuyển, số tầng lặp quá sâu làm bước tiến quá nhỏ, không thể nhanh chóng tiếp cận cận trên, sai số lớn và không qua được dữ liệu SPOJ. Nguyên nhân chính là hạn chế của số lần lặp và độ chính xác số thực.

**Một hướng khác.** Để tăng độ chính xác trong bài này, có thể dùng nhân ma trận. Lưu kết quả sau lần lặp thứ  $k$  trong một vector, rồi dùng ma trận chuyển để tính lần  $k + 1$ . Nhờ bình phương lặp, ta có thể thực hiện số vòng lặp cấp  $2^T$ . Cách này gọi tới nhân ma trận để tính bao đóng truyền được của đồ thị, còn SPFA trước đó giống dùng BFS để tính bao đóng truyền được. Dù bài là giải phương trình, khi dùng phương pháp lặp thì nó cùng mạch với thuật toán đồ thị: một bên truyền tính liên thông, một bên truyền giá trị lặp.

Mỗi phương pháp có ưu nhược điểm riêng. Điều này cũng cho thấy SPFA không phải vạn năng; khi áp dụng cụ thể cần cân nhắc nhiều mặt.

**Tiểu kết.** Dù kết quả chưa hoàn hảo, ta có một cách giải khác khử Gauss. Với những bài mà giá trị cuối của trạng thái phụ thuộc lẫn nhau, phương pháp lặp của SPFA có thể cho nghiệm khá chính xác dưới điều kiện thích hợp. SPFA còn có tính mở rộng mà khử Gauss không có: nếu phương trình phi tuyến, hoặc không chỉ gồm cộng trừ nhân chia mà còn có min, max, khử Gauss không còn áp dụng được, trong khi SPFA sau điều chỉnh vẫn có thể phát huy tác dụng. Nó giống tư tưởng giải phương trình bằng phương pháp lặp trong tính toán số: liên tục truyền giá trị trạng thái và tiến gần nghiệm cuối.

Tuy vậy, không được áp dụng máy móc. Bài Mario có tính đặc biệt: giá trị lặp đơn điệu không giảm và hệ số nhỏ hơn 1. Với bài tương tự khác, cần biến đổi và chứng minh cẩn thận trước khi dùng SPFA.

### 3.4 Một lớp quy hoạch động có chuyển trạng thái mang tính hậu hiệu

Trong phần trước ta đã bàn về SPFA để giải phương trình. Trong một số phương trình chuyển của quy hoạch động, đôi khi không tìm được thứ tự topo, và cách lặp truyền thống có thể tạo nghiệm không hợp

lệ. Khi đó cần tổng hợp các phương pháp đã nêu để tìm lời giải đúng.<sup>2</sup>

### Ví dụ 6: Apple

Trong một ma trận  $n \times m$ , một kẻ trộm táo bị bạn nhỏ  $Y$  phát hiện. Kẻ trộm rất khỏe nên  $Y$  không thể đuổi hắt đi, nhưng hắt cũng không muốn làm hại  $Y$  vì chỉ muốn táo. Vì thế  $Y$  quyết định hái táo trước kẻ trộm. Ma trận có ba loại ô: đất trống, cây táo và chướng ngại. Có đúng 4 cây táo, mỗi cây đúng 3 quả.  $Y$  và kẻ trộm có thể đứng trên đất trống hoặc cây táo, nhưng không đứng trên chướng ngại. Kẻ trộm đi trước, sau đó tới  $Y$ ; hai người luân phiên di chuyển cho tới khi không còn táo để hái.

Quy tắc hái táo:

1. Khi một người tới một ô cây táo còn táo, người đó lập tức nhận một quả.
2. Khi di chuyển, một người có thể chọn đứng lại trên ô cây táo để hái một quả nếu còn táo.

Mỗi lần, một người có thể đi từ  $(x, y)$  sang một trong bốn ô kề cạnh, nhưng ô đến không được là chướng ngại, ngoài ma trận, hoặc vị trí người kia đang đứng. Người đó cũng có thể đứng yên nếu đang ở ô cây táo, hoặc không thể đi tới ô kề nào. Giả sử cả  $Y$  và kẻ trộm đều chơi tối ưu, cần tính số táo tối đa kẻ trộm có thể lấy.

Đây là một bài trò chơi, quy mô nhỏ nên dễ nghĩ tới quy hoạch động. Đặt  $f[\text{Thief}, Y, \text{Apple}]$  là số táo tối đa kẻ trộm có thể lấy khi kẻ trộm và  $Y$  lần lượt ở hai vị trí đó, trạng thái táo được mã hóa cơ số 4, và tới lượt kẻ trộm. Đặt  $g[\text{Thief}, Y, \text{Apple}]$  là số táo tối thiểu kẻ trộm nhận được khi tới lượt  $Y$ . Phương trình chuyển rất đơn giản: lần lượt liệt kê hướng đi của từng người theo luật, nhận trạng thái kế tiếp, rồi  $f$  lấy lớn nhất,  $g$  lấy nhỏ nhất.

Nhìn qua tưởng như đã giải xong, nhưng phân tích thêm thấy vấn đề giống phần 3.1: khi trạng thái táo không đổi, chuyển giữa các trạng thái có thể tạo chu trình. Ta thử cách thông thường trước.

**Phân tích 1.** Vì là bài trò chơi, tồn tại cây quyết định; các cạnh trên cây là chiến lược tối ưu của từng người. Trong thực tế, trừ khi kẻ trộm không thể lấy thêm táo, trạng thái khi chơi sẽ không xuất hiện chu trình. Nếu có chu trình, nghĩa là hai người trở về chỗ cũ mà không ai thu được gì, điều đó vô nghĩa; đối quyết định còn có thể lấy được nhiều táo hơn.

Nhưng bài này không thể dùng cách gán nhãn thông thường, vì không biết nghiệm hiện tại đã là nghiệm cuối hay chưa.  $G$  có thể giảm,  $F$  có thể tăng, tương tự dùng Dijkstra trên đồ thị có trọng số âm.

Có thể dùng cách phần 3.1, tích lũy dần các giá trị tốt cục bộ để suy ra tối ưu hay không? Điểm mấu chốt là hướng tối ưu hóa của hai trạng thái kề nhau khác nhau.  $f$  lấy lớn nhất trong các  $g$ , còn  $g$  lấy nhỏ nhất trong các  $f$ . Giả sử ta đang có nghiệm tương đối tốt  $g'[]$  với  $g'[]$  lớn hơn nghiệm tối ưu  $g[]$ , rồi dùng  $g'[]$  cập nhật  $f[]$ . Theo cách cập nhật truyền thống, nếu sau đó tìm được  $g[]$ , ta lại thấy  $g[] < f[] = g'[]$  nên không cập nhật được. Điều này vô lý.

Vấn đề là không thể dựa vào việc tích lũy các nghiệm tương đối tốt để thu dần nghiệm tối ưu. Trong bài này, nghiệm tương đối tốt không có ý nghĩa thực tế và không hợp lệ.

So sánh với đường đi ngắn nhất: dù nghiệm hiện tại chưa tối ưu, mọi nghiệm hiện tại đều hợp lệ vì tương ứng với một đường đi hợp lệ. Bản chất sâu hơn là khi một trạng thái  $f(x)$  đã cập nhật trạng thái khác rồi sau đó chính nó lại được cải thiện, vì mọi trạng thái đều tối thiểu hóa, giá trị đã truyền ra trước đó chỉ trở thành thứ cấp và không ảnh hưởng tính đúng. Mỗi trạng thái cập nhật đơn điệu. Đây là tiền đề căn bản cho tính đúng của SPFA và Bellman–Ford, nhưng trong bài này tiền đề đó không còn; chuyển trực tiếp có hậu hiệu.

---

<sup>2</sup>Ở đây “hậu hiệu” chỉ hậu hiệu trong quá trình thực hiện chuyển, khác với cách nói thường gặp rằng hậu hiệu do tối ưu cục bộ trong phương trình gây ra.

Có thể có người muốn viết hệ ràng buộc hiệu:

$$F[X] \geq G[X'] + \text{Apple}, \quad G[Y] \leq F[Y'].$$

Nhưng rõ ràng  $F[] = 12, G[] = 0$  thỏa mọi bất đẳng thức. Sai lầm nằm ở chỗ kết quả của hệ ràng buộc không thật sự tương ứng với một nghiệm trò chơi; bất đẳng thức đơn lẻ không có ý nghĩa. Đây là lý do những bài trông giống nhau không nhất thiết dùng được cùng thuật toán: nhiều giả thiết thường bị mặc định trong lúc làm bài, và khi giả thiết không còn đúng thì thuật toán sai.

**Phân tích 2.** Nếu không thể dùng quan hệ chuyển để suy ra đúng nghiệm, có thể bỏ qua trở ngại đó và dùng trực tiếp trạng thái cuối bằng SPFA không? Ràng buộc hiệu ở trên thất bại vì chỉ xét quan hệ từng cặp trạng thái, bỏ qua liên hệ toàn cục, nên dấu bằng của bất đẳng thức không nhất thiết đạt được. Nếu bỏ khái niệm tối ưu cục bộ và xem bài toán trực tiếp là một hệ phương trình, nó đúng là loại hệ có min, max như phần 3.3. Ta có thể tiếp tục tư tưởng lặp ở đó.

Cụ thể, khởi tạo mọi giá trị bằng 0. Với mỗi phương trình chuyển, nếu phát hiện hai vế khác nhau thì gán lại. Cần chú ý phương trình chuyển ở đây, giống phần 3.3, phải được xét như một tổng thể. Ví dụ

$$F[X] = \text{Max}(G[Y'] + \text{Apple})$$

thì phải liệt kê mọi  $G[Y']$  liên quan, lấy giá trị lớn nhất rồi mới cập nhật. Khi truyền giá trị từ một đỉnh hoạt động  $x$  xuống  $u$ , ta vẫn phải đồng thời xét các đỉnh khác kề với  $u$ , thông qua hàm  $\text{Get}(u)$ . Cách này chưa bảo đảm nghiệm hiện tại luôn hợp lệ cuối cùng, nhưng bảo đảm mỗi giá trị hiện tại thật sự lấy từ các đỉnh kề và thỏa phương trình chuyển của chính nó.

```
for each edge (x, u) in E:
    if Get(u) != Now[u]:
        Now[u] = Get(u)
```

Cài đặt cụ thể rất giống phần 3.3, nhưng cần phân biệt với thuật toán truyền thống.

Tính đúng cần chứng minh. Khi thuật toán kết thúc, mọi phương trình đều được thỏa, nên nghiệm thu được là nghiệm hợp lệ và đúng. Còn lại chỉ cần chứng minh thuật toán dừng.

Xét giá trị trạng thái theo từng lớp từ nhỏ tới lớn. Ban đầu mọi trạng thái bằng 0, nên các trạng thái có giá trị cuối là 0 rõ ràng sẽ không tiếp tục bị cập nhật. Giả sử mọi trạng thái có giá trị cuối trong  $0..K$  đã được tính ra; từ đặc tính phương trình, phần này sẽ không đổi nữa. Với  $G[]$ , vì lấy nhỏ nhất nên giá trị chắc chắn lấy trong  $0..K$ . Với  $F[]$ , nếu giá trị cuối không quá  $K$ , các đỉnh kề liên quan cũng phải nằm trong  $0..K$ , nếu không giá trị cuối không thể không quá  $K$ . Phần giá trị  $K + 1..12$  có thể được suy dần ra, và một khi giá trị  $K + 1$  được xác định thì cũng không đổi. Tổng cộng chỉ có các giá trị từ 0 tới 12, nên thuật toán nhất định dừng.

Vì vậy tính đúng được chứng minh. Cài đặt rất đơn giản. Trong bài này, SPFA dùng DFS nhanh hơn BFS khoảng 25%.

Liệu chứng minh trên có cho ta thuật toán chuyển từng lớp theo giá trị trạng thái hay không? Nhìn qua có vẻ được, nhưng thực ra ta chỉ chứng minh trạng thái có giá trị cuối  $K$  nhất định sẽ đạt tới  $K$ ; trạng thái hiện tại bằng  $K$  chưa chắc đã là trạng thái cuối. Ta không phân biệt được nghiệm hiện tại nào hợp lệ, nên không thể phân lớp trạng thái theo cách đó.

**Tổng kết phần ứng dụng.** Sau khi có thuật toán cuối, ta thấy mình đã đi một vòng dài và cuối cùng lại dùng trực tiếp phương trình gốc, giải bằng một phương pháp lặp gần như cơ học. Nếu nhìn rõ bản chất, đây chính là tinh thần của SPFA: tìm mâu thuẫn trong lặp và giải mâu thuẫn trong lặp. Sự đơn giản cơ học cũng là ưu điểm của SPFA.

Phương pháp lập có thể xem là lời giải chung cho phần lớn bài toán phương trình và tối ưu. Phương pháp gán nhãn hay Bellman–Ford trên đồ thị có trọng số âm chỉ là các trường hợp đặc biệt. Trong thực hành, ta thường giản lược rất nhiều nhờ tính chất riêng của từng bài, điều đó là tốt; nhưng nếu không hiểu rõ điều kiện áp dụng, rất dễ dùng sai thuật toán.

Qua thảo luận trên, các bài thường gặp có thể chia thành ba loại:

#### **Loại thứ nhất**

Quy hoạch động không có chu trình; mô hình quan hệ là cây hoặc DAG. Ta giải theo quan hệ topo, không cần lặp nhiều lần.

#### **Loại thứ hai**

Thường là bài tối đa hoặc tối thiểu, trong đó quan hệ chuyển có chu trình và mô hình là đồ thị có chu trình. Các đỉnh không phụ thuộc lẫn nhau theo kiểu gây hậu hiệu, và việc truyền giá trị tối ưu không tạo hậu hiệu; phần lớn thỏa bất đẳng thức tam giác gốc. Ta có thể dùng cách giống đường đi ngắn nhất: so sánh và cập nhật các nghiệm tốt hơn để thu được toàn bộ giá trị tối ưu.

#### **Loại thứ ba**

Trạng thái không chỉ có chu trình mà còn có thể đan chéo nhau như phần 3.3, hoặc tính tối ưu không truyền trực tiếp qua đỉnh như phần 3.4. Khi đó phải liên tục tìm mâu thuẫn trong mạng quan hệ và gán lại.

Bản chất của cả ba loại đều thống nhất: nói lỏng. Loại thứ nhất tìm được một thứ tự để mỗi quan hệ chuyển chỉ cần gán một lần, nên hiệu quả cao. Đặc điểm bản chất là ta dễ biết trạng thái nào hiện đã tối ưu. Loại thứ hai khác ở chỗ có chu trình, ta không biết trạng thái nào hiện tối ưu, nhưng biết chắc có một số nghiệm đã tối ưu và nghiệm tốt hơn cũng hợp lệ, nên có thể truyền giá trị từ trạng thái hoạt động ra xung quanh. Loại thứ ba dành cho quan hệ tổng quát hơn; mỗi lần cập nhật phải bảo đảm giá trị hiện tại thỏa phương trình chuyển của chính đỉnh đó. Vì vậy, dù là bất đẳng thức hay phương trình, dù mục tiêu cuối là gì, nếu chứng minh được tính hợp lệ và hội tụ, ta có thể liên tục điều chỉnh để kết quả thỏa điều kiện ràng buộc và phát huy tác dụng của SPFA. Đổi lại, hiệu quả thường thấp hơn hai loại trước.

Các ví dụ trên đã minh họa nhiều mặt của vấn đề. Điều quan trọng không phải chỉ là thấy thuật toán cuối cùng đơn giản, mà là hiểu rõ phạm vi áp dụng của từng loại bài và bản chất của nó. Như vậy mới có thể thật sự dùng SPFA một cách linh hoạt.

## **4 Phụ lục**

### **4.1 Nguyên văn ý chính các bài ví dụ**

**Ví dụ 1: WordRings.** ACM-ICPC Central European 2005, liên kết gốc: <http://acm.pku.edu.cn/JudgeOnline/problem?id=2949>.

**Ví dụ 2: BIC, Olympic Tin học Baltic 2002.** Mạng đường cao tốc thu phí ở Byteland phát triển rất nhanh và trở nên dày đặc, khiến việc chọn tuyến tốt nhất trở thành vấn đề khó. Mạng gồm các đường hai chiều nối các thành phố; mỗi đường có thời gian di chuyển và phí phải trả. Một tuyến gồm các đường liên tiếp. Tổng thời gian là tổng thời gian các đường, tổng phí là tổng phí các đường. Tuyến càng nhanh và càng rẻ càng tốt. Chính xác hơn, một tuyến tốt hơn tuyến khác nếu nó nhanh hơn mà không đắt hơn, hoặc rẻ hơn mà không chậm hơn. Một tuyến nối hai thành phố được gọi là tối thiểu nếu không có tuyến nào tốt hơn. Có thể có nhiều tuyến tối thiểu không so sánh được, hoặc không có tuyến nào.

Trong ví dụ của đề, các đường giữa bốn thành phố có cặp số (phí, thời gian). Từ thành phố 1 tới 4, các tuyến 1 – 2 – 4 và 1 – 3 – 4 đều có phí 4, thời gian 5; tuyến 1 – 2 – 3 – 4 có phí 6, thời gian 4; tuyến

1 – 3 – 2 – 4 có phí 4, thời gian 10. Hai tuyến đầu tốt hơn tuyến cuối. Có hai cặp phí-thời gian tối thiểu: (4, 5) và (6, 4).

Nhiệm vụ: đọc mạng đường và hai thành phố đầu/cuối từ `bic.in`; tính số cặp phí-thời gian tối thiểu khác nhau nối hai thành phố đó, các tuyến có cùng cặp phí-thời gian chỉ tính một lần; ghi kết quả ra `bic.out`.

Dữ liệu vào: dòng đầu có  $n, m, s, e$ , với  $1 \leq n \leq 100$ ,  $0 \leq m \leq 300$ ,  $1 \leq s, e \leq n$ ,  $s \neq e$ . Mỗi dòng trong  $m$  dòng tiếp theo chứa  $p, r, c, t$ : hai đầu đường, phí  $c$ , thời gian  $t$ , trong đó  $0 \leq c, t \leq 100$ . Có thể có nhiều đường giữa cùng hai thành phố. Kết quả là một số nguyên: số cặp phí-thời gian tối thiểu từ  $s$  tới  $e$ .

| <code>bic.in</code> | <code>bic.out</code> |
|---------------------|----------------------|
| 4 5 1 4             | 2                    |
| 2 1 2 1             |                      |
| 3 4 3 1             |                      |
| 2 3 1 2             |                      |
| 3 1 1 4             |                      |
| 2 4 2 4             |                      |

**Ví dụ 3: Segments.** SPOJ, liên kết gốc: <http://www.spoj.pl/problems/SEGMENTS/>.

**Ví dụ 4: Kế hoạch tham quan.** Winter Camp 2008, liên kết gốc: <http://www.noi.cn/noi/showNews.jsp?newsId=100020000342>.

**Ví dụ 5: Mario.** SPOJ, liên kết gốc: <http://www.spoj.pl/problems/MARIOGAM/>.

**Ví dụ 6: Apple.** Bạn nhỏ  $Y$  có một rừng táo rất lớn. Vào mùa thu hoạch, một kẻ trộm muốn trộm táo và bị phát hiện. Kẻ trộm khỏe nên  $Y$  không đuổi được, nhưng hẳn không muốn làm hại  $Y$ .  $Y$  quyết định hái trước để giảm số táo kẻ trộm lấy được. Rừng táo được xem là ma trận  $5 \times 6$ , gồm đất trống, cây táo và chướng ngại. Có đúng 4 cây táo, mỗi cây đúng 3 quả.  $Y$  và kẻ trộm có thể đứng trên đất trống hoặc cây táo, không đứng trên chướng ngại. Kẻ trộm đi trước, sau đó hai người luân phiên cho tới khi không còn táo.

Quy tắc hái táo:

1. Khi một người tới ô cây táo còn táo, người đó lập tức lấy một quả.
2. Khi di chuyển, một người có thể đứng lại trên ô cây táo để hái một quả nếu còn táo.

Mỗi lần, một người có thể đi sang ô kề cạnh, nhưng không được tới tường, ra ngoài ma trận, hoặc tới vị trí người kia. Người đó có thể đứng yên nếu đang ở ô cây táo, hoặc không thể đi tới bất kỳ ô kề nào.

Đầu vào có nhiều bộ. Dòng đầu là  $T \leq 20$ . Mỗi bộ gồm 5 dòng, mỗi dòng 6 ký tự trong tập  $\{., \#, 3, S, T\}$ :  $.$  là đất trống,  $\#$  là chướng ngại, 3 là cây táo, S là  $Y$ , T là kẻ trộm. Mỗi bộ có đúng một S và một T; vị trí của họ là đất trống. Ít nhất một nửa số ô là chướng ngại, và sau mỗi bộ có một dòng trống. Với mỗi bộ, in ra số táo tối đa kẻ trộm lấy được.

Sample input

```
3
T3333.
#####S
#####
```

```
#####  
#####
```

```
####..  
.3.3..  
33.T.S  
#####  
#####
```

```
##3###  
##.##3  
TS3###  
##.###  
##3###
```

Sample output

```
9  
12  
0
```

Với mẫu thứ ba,  $Y$  có thể đứng mãi trên cây táo bên phải mình, khiến kẻ trộm không lấy thêm được táo nào; trò chơi kết thúc.

## 4.2 Chương trình nguồn và dữ liệu thử trong gói gốc

Các tệp đi kèm được liệt kê trong PDF gồm: Path\_Bfs.cpp, Path\_Greedy.pas, Path\_Limit.pas, Circle\_Limit.cpp, Word\_Rings.pas, Apple.cpp, Trip.pas, Segments.cpp, Mario.cpp, Bic.pas, Apple\_Data.rar và Bic\_data.rar.

## 5 Tài liệu tham khảo

1. Thomas H. Cormen và các tác giả, *Introduction to Algorithms*, ấn bản 2, Nhà xuất bản Công nghiệp Cơ khí.
2. Robert Sedgewick, *Algorithms in C++: Graph Algorithms*, ấn bản 3, Nhà xuất bản Đại học Thanh Hoa.
3. Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi Tin học*, Nhà xuất bản Đại học Thanh Hoa.
4. Guo Huayang, slide giảng bài Winter Camp 2008.
5. Amber, *Tổng kết lý thuyết đồ thị*.
6. Yu Yuanming, luận văn năm 2006, *Thuật toán đường đi ngắn nhất và ứng dụng*.
7. Feng Wei, luận văn năm 2006, *Sự kết hợp hoàn hảo giữa số và đồ thị: bàn sơ về hệ ràng buộc hiệu*.

## 6 Lời cảm ơn

1. Cảm ơn hai thầy hướng dẫn Song Xinbo và Qiu Chongzhi.

2. Cảm ơn hai huấn luyện viên đội tuyển quốc gia Tang Wenbin và Hu Weidong.
3. Cảm ơn CCF đã cung cấp nền tảng để mọi người cùng trao đổi và thảo luận.
4. Cảm ơn bạn Chen Qifeng của Đại học Khoa học và Công nghệ Hồng Kông đã giúp đỡ bài viết này.
5. Cảm ơn bạn Guo Huayang của Đại học Thanh Hoa đã cung cấp bài *Trip*.
6. Cảm ơn các thành viên hark, chenyxixi và những người khác trên PKU Online Judge đã cung cấp ý tưởng về cách tìm nhanh chu trình âm.
7. Cảm ơn bạn Feng Yi ở trường tôi đã đưa ra nhiều ý kiến chỉnh sửa cho luận văn.
8. Cảm ơn cha mẹ đã ủng hộ và quan tâm.
9. Cảm ơn các thầy cô, bạn học và bạn bè khác đã giúp đỡ; cảm ơn sự giúp đỡ nhiệt tình và sự ủng hộ của mọi người.